Hướng dẫn demo toàn trình: từ C đến CPU RISC-V trên Logisim

Nov 1, 2026 · @anhlmv

Một chương trình C đọc số n từ 8 công tắc, rồi in kết quả tính toán ra màn hình TTY, hiển thị fib(n) trên LED 7 đoạn và n trên 8 LED. Chương trình được biên dịch thành mã máy RV32IM và chạy trên chính CPU tự thiết kế.

1. Chuẩn bị

Cần Python 3.8+, Logisim-evolution 4.1.0 và gói `ziglang` (chứa clang + lld biên dịch được cho RISC-V, không cần cài GCC riêng).

1. Cài trình biên dịch (một lần):

   ```
   pip install ziglang
   python -m ziglang version
   ```

   Lệnh thứ hai in ra số phiên bản là cài đúng.
2. Giải nén `final_files.zip` đè vào thư mục `riscv32`. Sau khi giải nén có thêm:
   - `tools/cc.py`: công cụ biên dịch C thành file nạp ROM.
   - `c/`: `start.S` (điểm vào), `link.ld` (bản đồ bộ nhớ), `riscv_io.h` (hàm I/O), `rt.c` (`memset`, `memcpy`).
   - `demo/calc.c`: chương trình demo.
3. Mọi lệnh bên dưới chạy từ thư mục `riscv32`, ví dụ trong PowerShell:

   ```
   cd D:\HUST\Documents\Master\modern_computer_architecture\projects\riscv32
   ```

4. Kiểm tra circuit `main` của file CPU đã nối đủ thiết bị từ Lab 15: khối `IO`, LED Bar 8 đèn, 8 Hex Digit Display, TTY, DIP Switch 8 công tắc, và TTY nhận cùng clock với CPU.

Nên demo trên bản pipeline (`pipeline/riscv_pipe.circ`), vì nó có đủ mọi giai đoạn, kể cả dự đoán nhánh. Bản đơn chu kỳ chạy cùng file `.hex` y hệt.

2. Chương trình C

`demo/calc.c` là một "máy tính số học": mỗi khi công tắc thay đổi, nó tính lại và in một dòng mới.

	Thiết bị	Nội dung
Đầu vào	8 công tắc DIP	số n (0–255)
Đầu ra	TTY	`n=10 n!=3628800 fib=55 tong=55` (thêm `nguyen to` nếu n là số nguyên tố; `tran` khi kết quả vượt 32 bit)
Đầu ra	8 LED 7 đoạn	fib(n) dạng thập phân, hoặc `EEEEEEEE` nếu quá 8 chữ số
Đầu ra	8 LED	n dạng nhị phân

```
#include "riscv_io.h"

static unsigned fact(unsigned n) {              /* MUL; tra ve 0 neu tran 32
bit */
    unsigned r = 1;
    for (unsigned i = 2; i <= n; i++) {
        if (r > 0xFFFFFFFFu / i) return 0;      /* DIVU de kiem tra tran */
        r *= i;
    }
    return r;
}

static unsigned fib(unsigned n) {
    unsigned a = 0, b = 1;
    while (n--) { unsigned t = a + b; a = b; b = t; }
    return a;
}

static int is_prime(unsigned n) {               /* REMU */
    if (n < 2) return 0;
    for (unsigned d = 2; d * d <= n; d++)
        if (n % d == 0) return 0;
    return 1;
}
```

```
static void report(unsigned n) {
    unsigned f = fib(n), fa = fact(n);
    IO_LED = n;
    IO_HEX = (f <= 99999999u && n <= 47) ? to_bcd(f) : 0xEEEEEEEEu;
    puts_("n="); putu(n);
    puts_("  n!="); if (fa) putu(fa); else puts_("tran");
    puts_("  fib="); if (n <= 47) putu(f); else puts_("tran");
    puts_("  tong="); putu(n * (n + 1) / 2);
    puts_(is_prime(n) ? "  nguyen to\n" : "\n");
}

int main(void) {
    puts_("RISC-V Logisim - gat cong tac DIP\n");
#ifdef ONESHOT
    report(IO_SW & 0xFF);       /* ban kiem thu: in mot lan roi dung */
    return 0;
#else
    unsigned last = 0x100;
    for (;;) {                  /* ban demo: theo doi cong tac mai mai */
        unsigned n = IO_SW & 0xFF;
        if (n != last) { report(n); last = n; }
    }
#endif
}
```

Chương trình dùng thiết bị qua `c/riscv_io.h`. Mỗi thiết bị là một con trỏ `volatile` tới địa chỉ I/O, nên `IO_LED = n` biên dịch thành một lệnh `sw` vào 0x80000000. `volatile` bắt trình biên dịch giữ đủ mọi lần đọc/ghi, kể cả vòng chờ UART.

Tên	Địa chỉ	Dùng để
`IO_LED`	0x80000000	ghi 8 LED
`IO_HEX`	0x80000004	ghi 8 LED 7 đoạn
`IO_UART`	0x80000008	ghi = gửi 1 ký tự ra TTY; đọc = 1 khi sẵn sàng
`IO_SW`	0x8000000C	đọc 8 công tắc
`putc_`, `puts_`		in ký tự, chuỗi (chờ UART rồi ghi)
`putu`, `puti`		in số hệ 10 (dùng `%` và `/` → REMU, DIVU)
`to_bcd`		đổi số thành BCD để LED 7 đoạn hiện số thập phân

Các giai đoạn của CPU mà chương trình dùng tới: MUL/DIVU/REMU (giai đoạn 7), SB/LBU khi ghi và đọc bộ đệm ký tự của `putu` (giai đoạn 6), I/O (giai đoạn 8), và các vòng lặp được dự đoán nhánh (giai đoạn 9).

3. Biên dịch C thành mã máy

Một lệnh biên dịch toàn bộ và ghi file nạp ROM:

```
python tools/cc.py demo/calc.c
```

Kết quả mong đợi:

```
Da ghi rom/calc.hex: 364 word ROM (1456 byte)
  doan khoi tao RAM: 38 lenh (ghi 12 word du lieu), main bat dau tai 0xB0
  du lieu trong RAM: 48 byte tu 0x1000; ngan xep tu 0x2000 di xuong
```

`cc.py` làm 4 việc.

1. **Biên dịch và liên kết** bằng `zig cc` cho RV32IM, không dùng thư viện chuẩn, theo bản đồ bộ nhớ trong `c/link.ld`. Cờ quan trọng là `-mcpu=generic_rv32+m+relax`: `+m` cho phép dùng lệnh nhân/chia; `+relax` cho linker đổi lời gọi hàm `auipc + jalr` thành `jal`, vì CPU không có AUIPC.
2. **Đọc file ELF:** `.text` (mã lệnh) đi vào ROM, `.rodata` / `.data` (chuỗi ký tự, hằng số) thuộc RAM từ 0x1000.

3. **Sinh đoạn khởi tạo RAM.** CPU là kiến trúc Harvard: lệnh `lw` không đọc được ROM, và Ctrl+R xóa RAM. Vì vậy `cc.py` đặt ở đầu ROM một đoạn mã ghi từng word dữ liệu vào RAM (`lui t0` + `addi t0` + `sw t0`), rồi nhảy tới `_start`. Với `calc.c`, đoạn này dài 38 lệnh, ghi 12 word chứa các chuỗi `"n="`, `" fib="`…; chương trình chính bắt đầu ở 0xB0.
4. **Kiểm tra tập lệnh** từng word: nếu còn lệnh CPU không hỗ trợ (AUIPC, FENCE, ECALL…), công cụ báo địa chỉ và dừng. Sau đó ghi `rom/calc.hex` định dạng Logisim "v2.0 raw".

Bản đồ bộ nhớ của chương trình C.

Vùng	Địa chỉ	Nội dung
ROM	0x0000–0x00AF	Đoạn khởi tạo RAM (do `cc.py` sinh)
ROM	0x00B0–	`_start` (đặt `sp = 0x2000`, gọi `main`), rồi mã C
RAM	0x1000–	`.rodata`, `.data`, `.bss`
RAM	– 0x1FFF	Ngăn xếp, đi xuống từ 0x2000
I/O	0x80000000–0x8000000C	LED, HEX, UART, công tắc

DMEM chỉ dùng bit 2–11 của địa chỉ, nên 0x1000 rơi vào ô 0 của RAM và 0x1FFC là ô cuối.

Để biên dịch bản kiểm thử (in một lần rồi dừng), thêm macro `ONESHOT`:

```
python tools/cc.py demo/calc.c -D ONESHOT -o rom/calc_test.hex
```

4. Chạy thử bằng mô phỏng Python

Trước khi mở Logisim, chạy thử bằng mô hình tập lệnh `rvsim.py` để biết trước kết quả đúng. `--sw` là giá trị 8 công tắc:

```
python tools/cc.py demo/calc.c -D ONESHOT -o rom/calc_test.hex --run --sw 13
```

```
--- Chay thu (rvsim), SW = 0x0D: 838 lenh (dung tai vong lap) ---
TTY:
RISC-V Logisim - gat cong tac DIP
n=13  n!=tran  fib=233  tong=91  nguyen to
LED = 00001101   HEX = 00000233
```

Một vài giá trị nên thử khi demo:

Công tắc	Dòng TTY	LED 7 đoạn	Điểm đáng chú ý
7 (`00000111`)	`n=7 n!=5040 fib=13 tong=28 nguyen to`	`00000013`	tất cả kết quả đều vừa 32 bit
10 (`00001010`)	`n=10 n!=3628800 fib=55 tong=55`	`00000055`	fib và tổng trùng nhau
13 (`00001101`)	`n=13 n!=tran fib=233 tong=91 nguyen to`	`00000233`	13! vượt 32 bit → `tran` (phát hiện băng DIVU)
47 (`00101111`)	`n=47 n!=tran fib=2971215073 tong=1128 nguyen to`	`EEEEEEEE`	fib(47) vừa 32 bit nhưng quá 8 chữ số
255 (`11111111`)	`n=255 n!=tran fib=tran tong=32640`	`EEEEEEEE`	giá trị lớn nhất

Mỗi lần in một dòng mất khoảng 800–1300 lệnh, chủ yếu cho vòng chia 10 khi in số và vòng kiểm tra nguyên tố.

5. Chạy trên Logisim

1. Mở `pipeline/riscv_pipe.circ` bằng Logisim-evolution 4.1.0.
2. Nạp chương trình: nhấp đúp `IMEM` ở cây Design, chuột phải vào ROM → **Load Image…** → chọn `rom/calc.hex` (bản chạy liên tục, không phải `calc_test.hex`).
3. Quay về circuit `main`. Gạt công tắc DIP về giá trị ban đầu, ví dụ 7 = `00000111`: chọn công cụ Poke (bàn tay), bấm vào từng công tắc.
4. Chọn **Simulate → Auto-Tick Frequency** và chọn tần số cao nhất trong danh sách. Mỗi chu kỳ CPU là 2 tick, và mỗi dòng in tốn khoảng 1000 chu kỳ, nên ở tần số thấp sẽ phải chờ lâu.
5. Nhấn **Ctrl+R** (reset: PC về 0, xóa thanh ghi và RAM), rồi **Ctrl+K** để chạy tự động.
6. Quan sát:
   - TTY in `RISC-V Logisim - gat cong tac DIP`, rồi `n=7 n!=5040 fib=13 tong=28 nguyen to`.
   - LED 7 đoạn hiện `00000013`; 8 LED sáng theo `00000111`.
7. Gạt công tắc sang giá trị khác trong lúc CPU đang chạy. Sau vài giây, TTY in thêm một dòng, LED và LED 7 đoạn đổi theo. Chương trình chỉ in khi giá trị công tắc thay đổi.

8. Nhấn Ctrl+K lần nữa để dừng. Khi dừng, Ctrl+T đi từng nửa chu kỳ để xem lệnh ở từng tầng trên các Probe `Instr`, `Instr_D`, `Instr_E`, `Instr_M`, `Instr_W`.

Gợi ý khi quay video.

- Mở song song file `demo/calc.c` và cửa sổ Logisim, để người xem thấy dòng `IO_HEX = ...` trong C tương ứng với LED 7 đoạn.
- Dừng đúng lúc chương trình đang in để chỉ vào vòng chờ UART (`lw` đọc 0x80000008, rồi `sw` ký tự vào cùng địa chỉ) và chân `IOWrite` của CPU bật lên 1 (gắn thêm một Probe nếu muốn thấy rõ).
- Chạy cùng file `calc.hex` trên `single/riscv_single.circ` để cho thấy hai CPU chạy cùng một mã máy.

6. Kiểm chứng tự động từng chu kỳ

`cpu_test.py` nạp bản `ONESHOT` vào một bản sao của file `.circ`, chạy Logisim ở chế độ dòng lệnh và so mọi tín hiệu quan sát với mô hình ở từng chu kỳ. `--sw` đặt giá trị công tắc mà mô hình dùng để cấp `IORData`.

```
python tools/cpu_test.py rom/calc_test.hex --sw 13 --jar <logisim-evolution-4.1.0-all.jar>
python tools/cpu_test.py rom/calc_test.hex --sw 13 --pipe 16 --jar <logisim-evolution-4.1.0-all.jar>
```

Kết quả đã kiểm trên chính các file `.circ` cuối cùng của dự án:

CPU	Công tắc	Kết quả
Đơn chu kỳ	13	PASS, 840 chu kỳ
Pipeline + dự đoán nhánh	13	PASS, 983 chu kỳ, 78 stall, 30 flush
Pipeline + dự đoán nhánh	47	PASS, 1303 chu kỳ
Pipeline + dự đoán nhánh	200	PASS, 1802 chu kỳ

PASS nghĩa là phần cứng chạy mã do trình biên dịch sinh ra đúng từng chu kỳ, kể cả mọi lần ghi I/O. Đây là băng chứng mạnh hơn so với chỉ nhìn TTY, vì một lỗi tạm thời rồi tự sửa cũng bị phát hiện.

7. Viết chương trình C của riêng bạn

Bất kỳ file C nào tuân theo các quy tắc dưới đây đều biên dịch và chạy được bằng `python tools/cc.py <file>.c`.

Quy tắc.

- Bắt đầu bằng `#include "riscv_io.h"` và viết hàm `int main(void)`. Khi `main` trả về, CPU rơi vào vòng lặp dừng trong `start.S`.
- Không có thư viện chuẩn: không `printf`, `malloc`, `float`. In bằng `putc_`, `puts_`, `putu`, `puti`.
- Truy cập thiết bị qua `IO_LED`, `IO_HEX`, `IO_UART`, `IO_SW`. Thiết bị mới thì khai báo con trỏ `volatile`, nếu không trình biên dịch có thể bỏ qua lệnh đọc/ghi.
- Giới hạn bộ nhớ: mã tối đa 4 KB (1024 lệnh, gồm cả đoạn khởi tạo); dữ liệu cộng ngăn xếp tối đa 4 KB. Tránh mảng lớn trên ngăn xếp và đệ quy sâu.
- Phép nhân, chia, chia lấy dư trên `int` / `unsigned` 32 bit chạy trực tiếp bằng M-extension. Phép 64 bit (`long long`) sẽ cần hàm thư viện và bị báo lỗi khi liên kết.
- Nên thêm nhánh `#ifdef ONESHOT` chạy một lần rồi `return 0`, để kiểm bằng `cpu_test.py`.

Lỗi thường gặp.

Hiện tượng	Nguyên nhân	Cách xử lý
`No module named ziglang`	Chưa cài trình biên dịch, hoặc cài vào Python khác	`python -m pip install ziglang` bằng đúng lệnh `python` dùng để chạy `cc.py`
`lenh CPU khong ho tro` kèm địa chỉ	Mã dùng AUIPC, FENCE, ECALL… (ví dụ do inline assembly hoặc địa chỉ tương đối)	Bỏ đoạn mã đó; thử `-O 1` hoặc `-O s`
`undefined symbol: __udivdi3` (hoặc `__divdi3`, __umoddi3)	Phép tính 64 bit	Đổi sang kiểu 32 bit
`chuong trinh ... > ROM 1024 word`	Mã quá lớn	Dùng `-O s`, bỏ bớt chuỗi ký tự

Hiện tượng	Nguyên nhân	Cách xử lý
`--run` in đúng nhưng Logisim không in gì	TTY chưa nối clock, hoặc WE nối nhầm; hoặc chưa bấm Ctrl+R sau khi nạp ROM	Kiểm tra phần B của bước 15.3; Ctrl+R rồi Ctrl+K
TTY in ký tự rác	`.rodata` chưa vào RAM: nạp nhầm file `.hex` cũ hoặc tự dịch bằng lệnh khác	Luôn dịch bằng `cc.py` để có đoạn khởi tạo RAM
Gạt công tắc mà không thấy dòng mới	Logisim đang chạy chậm, dòng trước chưa in xong	Tăng Auto-Tick Frequency, chờ thêm vài giây